

COE67-222 Operating Systems - Lab 01 Report

Name: Taofix Yayueri

Student ID: 67104000

Observed output from GitHub Codespaces

- profiler.py: OS version = Linux 6.8.0-1064-azure, CPU cores = 2, Total RAM = 7.76 GB

- stress_test.py: จาก htop พบว่าโปรแกรมใช้ CPU หนักอยู่ที่ Core 0 เกือบ 100% โดยโปรเซส Python ใช้ CPU ประมาณ 99.3% ส่วน Core 1 ใช้งานประมาณ 37%

1. Hardware Baseline:

Based on lscpu and profiler.py, what is the exact OS version, number of CPU cores, and total RAM of your Codespace environment?

Answer:

Codespace ใช้ Linux kernel 6.8.0-1064-azure มี CPU แบบ logical 2 ตัว และมี RAM ทั้งหมด 7.76 GB ผลจาก profiler.py ตรงกับข้อมูลที่ดูจาก lscpu และ free -h

2. Observation:

During the CPU stress test (Phase 4), did your Python script utilize 100% of ALL available CPU cores, or just one? Explain what you saw in htop.

Answer:

จาก htop เห็นว่าโปรแกรมใช้ CPU หนักอยู่ที่ Core 0 เกือบ 100% โดยโปรเซส Python ใช้ CPU ประมาณ 99.3% ส่วน Core 1 ใช้งานประมาณ 37% จึงเห็นว่าโปรแกรมไม่ได้กระจายงานไปใช้ทุก core อย่างเต็มที่ เพราะโค้ดทำงานแบบ thread เดียว

3. Analysis:

In the context of Machine Learning, if you have a dataset of 500,000 images, why is it crucial to know your OS memory limits and CPU core counts before writing the training code?

Answer:

ถ้ามี dataset 500,000 รูป แล้วโหลดทั้งหมดเข้า RAM พร้อมกัน หน่วยความจำอาจไม่พอและทำให้ระบบช้าลงหรือ OS ต้องหยุดบางโปรเซส การรู้ขนาด RAM ช่วยให้แบ่งข้อมูลเป็นชุดเล็ก ๆ ได้ ส่วนจำนวน CPU cores ช่วยวางแผนการเตรียมข้อมูลและกำหนดจำนวน worker ให้เหมาะกับเครื่อง

Additional Summary:

Lab นี้ทำให้เห็นว่าโค้ด Python ต้องพึ่งทรัพยากรที่ OS จัดสรรให้ การดูข้อมูลจาก lscpu, free และ htop ช่วยให้รู้ข้อจำกัดของเครื่องก่อนเริ่มงาน ML ขนาดใหญ่

COE67-222 Operating Systems - Lab 02 Report

Name: Taofix Yayueri

Student ID: 67104000

Observed output from GitHub Codespaces

- Sequential Time: 9.81 seconds
- Multithreading Time: 9.20 seconds
- Multiprocessing Time: 8.41 seconds
- htop: Multithreading ใช้ CPU ได้ไม่เต็มทุก core ส่วน Multiprocessing กระจายงานไปหลาย core ได้ชัดเจนกว่า

1. Execution Time Comparison:

- Sequential Time: 9.81 seconds
- Multithreading Time: 9.20 seconds
- Multiprocessing Time: 8.41 seconds

2. System Observation (htop):

Explain the difference in CPU core utilization (in htop) when running the Multithreading script vs. the Multiprocessing script.

Answer:

ตอนใช้ Multithreading งานยังทำอยู่ในโปรเซสเดียวกัน จึงเห็นการใช้ CPU กระจุกอยู่ที่ core เดียวหรือสลับไปมา แต่ไม่ได้ทำงานหลาย core ได้เต็มที่ ส่วน Multiprocessing แยกงานเป็นหลายโปรเซส OS จึงส่งแต่ละโปรเซสไปทำงานบนหลาย core ได้พร้อมกัน

3. Analysis (The GIL):

Based on your results, why did Multithreading fail to make the code run faster? Why is Multiprocessing a better choice for CPU-bound tasks (like Machine Learning data augmentation) in Python?

Answer:

งานใน Lab เป็นงานคำนวณที่ใช้ CPU หนัก แม้ Multithreading จะใช้เวลา 9.20 วินาที ซึ่งเร็วกว่า Sequential เล็กน้อย แต่ไม่ได้ทำให้ใช้หลาย core ได้เต็มที่ เพราะ Python มี GIL และยังมีเวลาที่ใช้ในการสลับ thread ผลที่เร็วขึ้นอาจเกิดจากความคลาดเคลื่อนของการวัดเวลา ส่วน Multiprocessing เหมาะกับงานคำนวณมากกว่า เพราะแต่ละโปรเซสมิ GIL แยกกันและสามารถใช้หลาย core ได้จริง

Additional Summary:

ผลที่เห็นคือจำนวน thread ไม่ได้แปลว่าจะใช้ CPU ได้เต็มทุก core เสมอไป ต้องเลือกวิธีให้เหมาะกับงาน โดยงานคำนวณเหมาะกับ multiprocessing มากกว่า threading ใน Python

COE67-222 Operating Systems - Lab 03 Report

Name: Taofix Yayueri

Student ID: 67104000

Observed output from GitHub Codespaces

- race_condition.py: Expected Weight Value = 200 แต่ Actual Weight Value = 100 จึงพบ Race Condition
- ipc_pipe.py: Trainer รอข้อมูลจาก DataLoader แล้วได้รับ Image_Tensor_Batch_01 ผ่าน pipe สำเร็จ
- producer_consumer.py: Producer โหลด Image_Batch_0 ถึง Image_Batch_9 และ Consumer ดึงไป train ครบทุกชุด โดยไม่มีการอ่านช่องว่างหรือเขียนทับข้อมูลที่ยังไม่ถูกใช้

1. The Race Condition (Phase 2):

In the race_condition.py script, what exactly causes the final weight to be less than the expected value? Explain the role of the OS Scheduler in this error.

Answer:

ปัญหาเกิดจากการเพิ่มค่าไม่ได้ทำในคำสั่งเดียว แต่แบ่งเป็นอ่านค่าเดิม แล้วบวกหนึ่ง และเขียนค่ากลับ ถ้า Thread หนึ่งอ่านค่าแล้วถูก OS Scheduler สลับไปให้อีก Thread อ่านค่าเดิม อีก Thread จะเขียนค่าทับกลับมา ทำให้การเพิ่มบางรอบหายไป จึงได้ค่าต่ำกว่า 200

2. Mutex vs. Semaphore (Phase 3):

In producer_consumer.py:

- What is the specific job of the mutex?
- What is the specific job of the empty and full semaphores?

(Why can't we just use a Mutex for everything?)

Answer:

mutex ใช้กันไม่ให้ Producer กับ Consumer เข้าแก้ buffer และ index พร้อมกัน ส่วน empty ใช้นับช่องว่างที่ยังใส่ข้อมูลได้ และ full ใช้นับจำนวนข้อมูลที่พร้อมให้ Consumer อ่าน Mutex อย่างเดียวบอกได้ว่าใครเข้า critical section ได้ แต่ไม่ได้บอกว่าตอนนี้ buffer เต็มหรือว่าง จึงต้องใช้ Semaphore ช่วยควบคุมจำนวนช่องและลำดับการรอดด้วย

3. Deadlock Awareness:

In the Producer function, we wrote:

```
empty.acquire()
```

```
mutex.acquire()
```

What would happen if we swapped these two lines? (mutex first, then empty). Imagine the buffer is completely full. Explain the potential OS problem.

Answer:

ถ้า buffer เต็ม Producer จะถือ mutex แล้วไปหยดรอ empty ทำให้ Consumer ที่ต้องใช้ mutex เพื่อเอาข้อมูลออกเข้าไม่ได้ ทั้งสองฝ่ายจึงรอกันเอง Producer รอช่องว่าง ส่วน Consumer รอ mutex สถานการณ์นี้ทำให้ pipeline ค้างแบบ deadlock

Additional Summary:

Lab นี้ทำให้เห็นว่าการทำงานพร้อมกันต้องควบคุมทั้งการเข้าถึงข้อมูลและจำนวนข้อมูลใน buffer การใช้ mutex คู่กับ semaphore ช่วยให้ DataLoader กับงาน train ทำงานต่อเนื่องโดยไม่ทำข้อมูลเสียหรือรอกันไม่จบ

COE67-222 Operating Systems - Lab 04 Report

Name: Taofix Yayueri

Student ID: 67104000

Observed output from GitHub Codespaces

- deadlock_simulation.py: Model A ถือ GPU 0 แล้วรอ GPU 1 ขณะที่ Model B ถือ GPU 1 แล้วรอ GPU 0 ทำให้โปรแกรมค้าง
- deadlock_avoidance.py: ทั้งสองโมเดลขอ GPU 0 ก่อน แล้วจึงขอ GPU 1 ทำให้ทำงานจบได้
- bankers_algorithm.py: ระบบอยู่ใน safe state และได้ลำดับ Job_1 -> Job_2 -> Job_0
- starvation_sim.py: Greedy_Model_A ได้ resource 24091 ครั้ง, Greedy_Model_B ได้ 24052 ครั้ง ส่วน Polite_Model ได้ 297 ครั้ง
- deadlock_detection.py: Watchdog ตรวจพบ deadlock แล้ว Model A ปลอม GPU 0 ทำให้ Model B ทำงานต่อและจบงานได้

1. The Deadlock (Phase 2):

In the deadlock_simulation.py script, explain step-by-step how the Circular Wait condition occurred between Model A and Model B.

Answer:

Model A ขอและถือ GPU 0 ไว้ก่อน แล้วรอ GPU 1 ส่วน Model B ขอและถือ GPU 1 ไว้ก่อน แล้วรอ GPU 0 ทั้งสองตัวจึงถือ resource ที่อีกฝ่ายต้องการอยู่ ทำให้ไม่มีตัวไหนไปต่อหรือปล่อย resource ได้ เกิดเป็น Circular Wait และโปรแกรมค้าง

2. The Solution (Phase 3):

By changing the code in deadlock_avoidance.py so that Model B also requests GPU 0 first, how did this specifically break the Circular Wait condition?

Answer:

เมื่อทั้ง Model A และ Model B ขอ GPU 0 ก่อน ลำดับการขอ resource จะเหมือนกัน Model หนึ่งจะได้ GPU 0 และขอ GPU 1 จนทำงานเสร็จก่อน อีกตัวจึงค่อยได้ resource ต่อ ไม่มีกรณีที่แต่ละตัวถือ GPU คนละตัวแล้วรอกัน จึงตัด Circular Wait ออกไป

3. Starvation vs Deadlock (Phase 4):

Based on the results from starvation_sim.py:

- How is the final output of the Starvation script fundamentally different from the Deadlock script you ran in Phase 2?
- In your own words, explain how a process scheduler could use a technique called Aging to help the Polite_Model get a fair share of CPU time.

Answer:

Deadlock ทำให้ทั้งสองงานค้างและไม่มีการไหนทำต่อได้ ส่วน Starvation ระบบยังทำงานอยู่ แต่ Polite_Model ได้ใช้ resource น้อยกว่าเพราะ Greedy_Model ขอซ้ำบ่อยกว่า Scheduler สามารถใช้ Aging โดยค่อย ๆ เพิ่ม priority ให้โปรเซสที่รอนาน เมื่อ priority สูงพอ Polite_Model ก็จะได้คิวทำงาน ไม่ถูกแย่งตลอดเวลา

4. Deadlock Detection & Recovery (Phase 3.3):

- How did the timeout parameter simulate the Deadlock Detection mechanism of an Operating System?
- When the Deadlock was detected, what exact step did Model A take to perform Recovery and allow the rest of the system to continue?

Answer:

timeout กำหนดเวลาสูงสุดที่ Model A จะรอ GPU 1 ถ้าเกินเวลานี้แสดงว่างานไม่มีความคืบหน้าและอาจติด deadlock จากนั้น Model A ปลอย GPU 0 ที่ตัวเองถืออยู่ เพื่อให้ Model B หรือ GPU 0 ได้ resource ไปทำงานต่อ ระบบจึงไม่ค้างทั้งระบบ

Additional Summary:

Lab นี้ทำให้เห็นว่าการจัดลำดับการขอ resource ช่วยป้องกัน deadlock ได้ตั้งแต่ต้น แต่ถ้าควบคุมลำดับไม่ได้ OS ยังต้องตรวจจับการรอนานและทำ recovery รวมถึงดูแลความเป็นธรรมเพื่อไม่ให้บางงาน starvation